

Analog Interfacing: Quantization, Sampling, and the ADC

The world is continuous and your MCU is not. Two lossy conversions, their error budgets, and the peripheral that performs one of them

Textbook: Dean, *Embedded Systems Fundamentals*, 2nd ed., Chapter 6, printed pages 167–202.

Lab manual: Lab 07 (ADC and CMSIS DSP), Section 7.2–7.5. Your own Lab 07 report.

1 What we are actually after this week

Back in Week 01 the hot plate needed a temperature, and we said "the MCU may need to convert this signal into a form it can use" and moved on. This is the week we stop moving on.

The question: **a temperature sensor outputs 0.05 V per °C, so 10.000 °C is 0.5 V and 10.001 °C is 0.50005 V. Can your MCU tell those apart?**

The answer is "it depends", and unpacking what it depends on is the chapter. There are exactly two ways a conversion loses information, and each has a quantifiable error:

1. **Quantization**: the value axis is chopped into discrete levels. Error depends on *resolution*.
2. **Sampling**: the time axis is chopped into discrete instants. Error depends on *sample rate*, and getting it wrong produces a failure mode far nastier than mere inaccuracy.

Then the plan continues:

3. The **DAC**, briefly, because it is the same idea running backwards and your syllabus wants it as an activity.
4. **ADC architectures**: flash and successive approximation, and why nearly every MCU uses the second.
5. **Sample and hold**, single-ended versus differential, and triggering.
6. The **ADC peripheral** itself: enabling it, clocking it, channels, sampling time, resolution, and the conversion-time arithmetic.

Item 1 produces a guaranteed calculation question. Item 2 produces the concept question everyone half-remembers. Item 6 is Lab 07, and since you have already done Lab 07, this week is partly an explanation of why your own code worked.

2 Deep dive 1: Quantization, chopping the value axis

Start with the problem in its purest form. You want your MCU to drive a speaker with a sine wave. Recall from Week 02 that a GPIO output has exactly two levels, near ground and near V_{DD} .

So your "sine wave" is a square wave. That is **two-level quantization**, and it sounds like a buzzer because it is one.

Give yourself eight levels instead and the reconstruction starts to resemble a sine. Give yourself 4096 and it is a sine as far as any ear is concerned. The number of discrete values

available is the **resolution**, and more of them is better.

Quantization

The process of selecting one of multiple discrete values to represent an analog value that could, in principle, take infinitely many values.

The crucial consequence: **each quantized output value represents a range of input values**. Two inputs inside the same range produce the same code and are therefore **indistinguishable** to the MCU. Not approximately equal. Identical, with no way to recover the difference.

Two **voltage references**, V_{ref+} and V_{ref-} , set the boundaries of the conversion range. Usually the positive supply rail (3.3 V) and ground.

Because digital electronics are binary, the number of levels is a power of two. With B bits of resolution you get 2^B levels. Eight-bit resolution gives $2^8 = 256$.

2.1 The arithmetic

Take a two-bit example, $B = 2$, over 0 V to 1 V. Four levels, so each covers 0.25 V:

Input voltage	Decimal	Binary	Note
$> 1 \text{ V}$	—	—	Out of range
0.75 V to 1.00 V	3	11	
0.50 V to 0.75 V	2	10	0.51 V and 0.74 V both give 10
0.25 V to 0.50 V	1	01	
0.00 V to 0.25 V	0	00	
$< 0 \text{ V}$	—	—	Out of range

The bounds of the range that code n represents:

$$V_{n_lower} = \frac{n}{2^B}(V_{ref+} - V_{ref-}), \quad V_{n_upper} = \frac{n+1}{2^B}(V_{ref+} - V_{ref-})$$

For $n = 1$, $B = 2$, range 0 to 1 V: $V_{lower} = \frac{1}{4}(1) = 0.25 \text{ V}$ and $V_{upper} = \frac{2}{4}(1) = 0.50 \text{ V}$. The centre of that range is 0.375 V, which is *not* $n/2^B = 0.25 \text{ V}$, and that offset is a real bias.

Some converters correct it by shifting the range half a step down, so that code n is centred on $n/2^B$:

$$V_{n_lower} = \frac{n - 1/2}{2^B}(V_{ref+} - V_{ref-}), \quad V_{n_upper} = \frac{n + 1/2}{2^B}(V_{ref+} - V_{ref-})$$

Now $n = 1$ spans 0.125 V to 0.375 V, centred on 0.25 V. The trade is that the top and bottom codes become half-width.

And the **transfer function**, which converts a voltage to a code and is what the datasheet gives you:

$$n = \text{round} \left(2^B \cdot \frac{V_{in} - V_{ref-}}{V_{ref+} - V_{ref-}} \right)$$

Quantization error

As B increases, the gap between V_{n_lower} and V_{n_upper} shrinks, so quantization gets more accurate.

The step size, or **LSB**, is $\Delta V = \frac{V_{ref}}{2^B}$.

The **maximum quantization error is half a step**: $\epsilon_{max} = \pm \frac{\Delta V}{2} = \pm \frac{V_{ref}}{2^{B+1}}$.

Half, because the worst case is an input sitting exactly at the boundary between two codes, and the reported value is then off by half a step in one direction or the other.

In the lab

Your own Lab 07 report worked these numbers, and they are worth reproducing because they are the exam answer. At $V_{ref} = 3.3$ V:

Resolution	Levels	Step ΔV	Max error
8-bit	256	12.89 mV	± 6.45 mV
10-bit	1024	3.22 mV	± 1.61 mV
12-bit	4096	0.806 mV	± 0.403 mV

And the explanation you gave, which is the right one: each extra bit doubles the number of levels, halving the step size and therefore halving the maximum error, so $\Delta V \propto 2^{-B}$. Going from 8 to 12 bits multiplies the levels by 16 and divides the error by 16.

Now go back to the question that opened this section. A 12-bit ADC at 3.3 V resolves 0.806 mV. Our temperature sensor gives 0.05 V per $^{\circ}\text{C}$, so 0.001 $^{\circ}\text{C}$ is 0.05 mV, which is **sixteen times smaller than one LSB**. So no, a 12-bit ADC cannot distinguish 10.000 $^{\circ}\text{C}$ from 10.001 $^{\circ}\text{C}$. It would need about 16 more bits, or a sensor with much more gain, or amplification before the ADC. *That* is the answer to the chapter's opening question, and notice it is a design conclusion rather than a fact to memorise.

Beware the trap

Two things people get wrong here.

First, **resolution is not accuracy**. Resolution is how finely the ADC divides its range. Accuracy also involves reference voltage error, gain error, offset error, nonlinearity, and noise. A 12-bit ADC with a sloppy reference may deliver 9 bits of trustworthy information. The datasheet lists these separately for exactly this reason.

Second, watch the divisor. The step size uses 2^B , but the maximum code is $2^B - 1$, so converting a code back to a voltage uses $D = \frac{V_{in}}{V_{ref}}(2^B - 1)$, as your own report wrote it. Mixing up 2^B and $2^B - 1$ is a one-LSB error, which is exactly the kind of thing that costs a mark and is invisible on hardware.

3 Deep dive 2: Sampling, chopping the time axis

Better resolution fixes the value axis. Now time.

Sampling

A **sampled signal** is a discrete-time representation, a series of individual samples, of a continuous-time signal. The **sampling rate** f_s determines how often an input is measured or an output updated.

Any information between the samples is lost. Note that each sample may still be an analog value until it is quantized; sampling and quantization are separate operations that happen to occur together.

Sample a sine wave slowly and the reconstruction is poor. Sample faster and it improves. Fine so far, and it sounds like the same story as resolution: more is better, gradually.

It is not the same story, and here is where it gets interesting.

3.1 Aliasing, and why undersampling is qualitatively worse

Think about the signal in the frequency domain. A signal's **spectrum** plots power against frequency, and it is symmetric about 0 Hz. Say the highest frequency present is f_h .

Sampling at frequency f_s does something specific and slightly surprising: it **makes copies of the spectrum centred at every multiple of f_s** .

Now, if those copies are far enough apart, no harm done. But if f_s is too low, adjacent copies **overlap**, and the overlapping parts add together. Frequencies that were above $f_s/2$ in the original show up, in the sampled data, masquerading as *lower* frequencies.

The Nyquist criterion, and aliasing

Nyquist: if a signal has any energy at $f_s/2$ or above, those components appear in the sampled data at incorrect, lower frequencies.

Aliasing: distortion of a signal resulting from sampling at too low a frequency.

So the requirement is $f_s > 2f_h$, which your own Lab 07 report wrote as $f_s \geq 2f_{max}$.

Beware the trap

This is the point worth carrying away, so let me put it sharply.

Insufficient **resolution** gives you a *slightly wrong* answer, and the error is bounded by half an LSB. Annoying, predictable, and you can budget for it.

Insufficient **sample rate** gives you a *confidently wrong* answer, with no indication anything is amiss. A 300 Hz vibration sampled at 500 Hz appears in your data as a clean, plausible 200 Hz signal. Nothing in the samples reveals the deception, and no amount of filtering *afterwards* can undo it, because the information needed to separate real 200 Hz from aliased 300 Hz was destroyed at the moment of sampling.

Aliasing is not an inaccuracy, it is a fabrication. That asymmetry is why the two error sources get different treatment, and it is a very likely exam question.

3.2 The two defences

You prevent aliasing two ways, and you use both:

1. **Choose f_s well above $2f_h$.** Not just above, well above.
2. **Fit an analog low-pass anti-aliasing filter** before the ADC, to remove or heavily attenuate everything above f_h .

Note the ordering dependency: the filter must be **analog** and must sit **before** the converter. A digital filter running on your samples is too late, because the aliasing already happened.

And note why the two defences interact. A real low-pass filter does not cut off sharply, it rolls off over some band. The more headroom you leave between f_h and $f_s/2$, the gentler the roll-off you can accept, and the cheaper and simpler the filter. This is why f_s is often chosen at four or

ten times f_h rather than the theoretical minimum of two: **you are buying filter simplicity with sample rate.**

In the lab

Your Lab 07 Task 2 is a nice illustration of the distinction, though it is easy to miss. You fed a sine wave of 1 Hz to 100 Hz into PA1 and applied a 10-point moving average using `arm_mean_f32`. Your report notes that as the input frequency rose, the filtered amplitude progressively fell, confirming low-pass behaviour.

That is a **digital** filter, running after sampling. It attenuates high frequencies *in your samples*, which is genuinely useful for smoothing sensor noise. What it cannot do is protect you from aliasing, because anything that aliased did so before your filter ever saw it. If you had fed in a 3 kHz sine while sampling at, say, 1 kHz, the moving average would have obediently smoothed a signal that was never there.

Worth trying if you have bench time: raise the generator frequency well past half your sampling rate and watch the "frequency" your plot reports. It will go back down. That is aliasing, and seeing it once makes it permanent knowledge.

4 Deep dive 3: The DAC, the same idea backwards

Briefly, since your syllabus lists it as an activity rather than core material.

A **digital-to-analog converter** generates an analog output from a digital input value. Output may be a voltage or a current; we care about voltage DACs. As with the ADC, lower and upper reference voltages define the output range, and the lower one is usually just grounded at 0 V.

Everything from Section 1 applies unchanged. Resolution B gives 2^B output levels, the step is $V_{ref}/2^B$, and the transfer function runs the other way. Everything from Section 2 applies too: a DAC updated at f_s produces a stepped output, and reconstructing a smooth waveform needs a *reconstruction* low-pass filter on the output, which is the mirror image of the anti-aliasing filter on the input.

Book board vs your board

Book: the STM32F091RC has a DAC, used in the chapter for an analog waveform generator example.

Yours: the STM32F303VCT6 has **2 DAC channels**, and also **7 analog comparators** and **4 operational amplifiers** on-chip. The op-amps are genuinely useful and often overlooked: they can amplify a small sensor signal *before* it reaches the ADC, which is precisely the fix for the temperature-sensor resolution problem in Section 1.

The book also covers the analog comparator in this chapter, which your syllabus omits. Worth knowing it exists: a comparator is effectively a **1-bit ADC** that outputs which of two voltages is larger, and it can raise an interrupt on a threshold crossing without the CPU polling anything. For a "has this voltage gone too high" question, a comparator is far cheaper than an ADC.

5 Deep dive 4: How an ADC is actually built

Two architectures. Understanding both is how you understand the speed-versus-cost trade in every ADC datasheet.

5.1 Flash: fast and expensive

Recall that a comparator tells you which of two voltages is larger, making it a 1-bit ADC. So build a B -bit ADC out of 2^B comparators in parallel, each with a different reference voltage, and encode their outputs with digital logic.

Conversion time is just the comparator delay plus the encoder delay. Extremely fast, hence the name.

The catch is in the exponent. **Each additional bit doubles the number of comparators.** A 12-bit flash ADC needs 4096 of them, which is absurd in power, silicon area, and therefore cost. Flash architectures live in gigasample-per-second instruments, not microcontrollers.

5.2 Successive approximation: the one you actually have

Use *one* comparator and make a series of comparisons, changing its reference each time. The hardware is a **successive approximation register (SAR)**, a DAC, a comparator, and control logic. The algorithm is a **binary search**.

Trace it for a 6-bit conversion:

Step	SAR guess	Question	Answer
0	000000	—	Start, all bits clear
1	100000	Input above this?	Higher, so keep the 1
2	110000	Above this?	Lower, so clear it to 0
3	101000	Above this?	Lower, clear
4	100100	Above this?	Higher, keep
5	100110	Above this?	Higher, keep
6	100111	Above this?	Lower, clear
	100110	Final code	

The trade, stated cleanly

Flash: 2^B comparators, **one** time step. Cost grows exponentially with resolution, time is constant.

SAR: **one** comparator, B time steps. Cost grows barely at all, time grows linearly with resolution.

For a microcontroller, where a megasample per second is plenty and die area is money, SAR wins overwhelmingly. Which is why **a lower-resolution setting on your ADC is genuinely faster:** fewer bits means fewer binary-search steps. That is not a marketing quirk, it is the architecture.

Other architectures exist, sigma-delta and dual-slope integrating among them, and the book explicitly sets them aside.

6 Deep dive 5: Getting the signal in, and starting the conversion

Three practical mechanisms around the converter.

6.1 Input multiplexer

ADCs usually include an input mux so one converter can serve many channels, selected by a channel code in a control register. Recall the pin multiplexer from Week 03: same idea, same motivation, one expensive resource shared across many pins.

Note the consequence: a single ADC converting several channels does so **sequentially**, not simultaneously. If you need two voltages sampled at the same instant, one ADC cannot give you that.

6.2 Sample and hold

Here is a problem specific to SAR. The binary search takes B steps, and it is comparing against the input the whole time. **If the input moves during the conversion, the answer is wrong**, because earlier bits were decided against a different voltage than later ones.

The fix is a **sample and hold** circuit: conceptually a switch and a capacitor.

- **Switch closed (sample)**: the capacitor charges toward the input voltage.
- **Switch open (hold)**: the capacitor is disconnected and holds its voltage steady for the converter to work against.

And a detail with real consequences: the capacitor does **not** charge instantly. Its charging rate is limited by the capacitance and the resistance of both the switch and *the signal source*. So the switch must stay closed for a **minimum sample time**.

Beware the trap

That "resistance of the signal source" clause is the practical bite. A **high-impedance source needs a longer sample time**, because it charges the hold capacitor slowly.

So if you connect a high-value potentiometer or a sensor with a large output resistance and leave the sample time at its minimum, you get readings that are systematically low and that drift depending on what you measured previously, because the capacitor never fully reached the input voltage. The reading is wrong in a way that looks like a bad sensor.

This is why the lab manual tells you to choose sampling time "based on the input signal frequency" and why the fix for a suspiciously sluggish or biased ADC reading is usually *increase SMP*, not rewrite the code.

6.3 Single-ended versus differential

Two ways to represent a voltage

Single-ended: the information is the difference between the signal and **ground**.

Differential: the information is the difference between **two signals**, neither of which is ground.

Differential rejects noise far better, because interference picked up along the way tends to affect both wires equally and therefore cancels in the difference.

An ADC with differential support measures the difference directly, in one step. One without it must convert both signals separately and subtract in software, which is slower and, importantly, samples them at *different instants*.

6.4 Triggering

A **trigger** tells the ADC to sample, hold, and convert. Two kinds:

- **Software trigger:** write to a control register. Typically from code, but the DMA controller can also do it.
- **Hardware trigger:** a signal asserted by a circuit, inside or outside the MCU. Classically a **timer** generating a pulse every millisecond.

When conversion completes, the ADC sets a flag in a status register and can raise an interrupt request. The result waits in a data register.

Beware the trap

Software triggering has a defect that matters enormously and is easy to miss: **the sample interval is only as regular as your code.**

If your main loop triggers a conversion each pass, then every interrupt, every branch, and every variable-length operation jitters the spacing between samples. Recall Week 02's point about f_s : your entire aliasing analysis assumed a *constant* f_s , and now you do not have one.

For anything doing signal processing, **trigger the ADC from a hardware timer**, not from software. The timer's period is exact regardless of what the CPU is doing. Your Lab 07 used continuous conversion mode with an interrupt on completion, which is a reasonable middle ground, and a timer trigger is the version you would want for a real measurement system. This is also the correct answer if you are asked how to guarantee a uniform sample rate.

7 Deep dive 6: The ADC peripheral, concretely

Basic operation is four steps: select a channel with the mux, await a trigger, await completion, process the result. Recall the four phases from Week 03, because this is the same shape.

7.1 Enabling it, and the one non-obvious step

Recall from Week 03 that every peripheral needs its clock gated on first. The ADC adds a wrinkle.

Listing 1: The ADC needs time to stabilise, and it tells you when it is ready.

```
1 | ADC1->CR |= ADC_CR_ADEN;           /* enable */
2 | while ((ADC1->ISR & ADC_ISR_ADRDY) == 0) /* wait for ready */
3 |     ;
```

The five fields of ADC_CR:

Field	Write 1 to
ADEN	Enable the ADC. Then poll ADRDY in ADC_ISR until it is set
ADDIS	Disable and power down
ADSTART	Allow conversions in response to triggers
ADSTP	Stop conversions
ADCAL	Start calibration, for high-precision use

Beware the trap

Analog peripherals need warm-up time and digital ones do not. A GPIO port is usable the instant its clock arrives. An ADC contains bias circuits and a reference that must settle, so it publishes a ready flag and you must wait for it.

Skip the `ADRDY` wait and your first several conversions return garbage. Since they are usually thrown away during debugging anyway, this produces the delightful bug where the ADC "works fine except the first reading is always wrong".

The `ADCAL` calibration is also worth running at startup for anything precision-related. It compensates for offset error, which is one of the accuracy terms that resolution does not cover.

7.2 References and the conversion clock

The references are V_{DDA} , the analog supply, nominally 3.3 V, and V_{SSA} , analog ground.

The conversion clock `ADC_CLK` sets how fast sampling and conversion proceed, and it has a *legal range*, not just a maximum:

Quantity	Book's F091
f_{ADC} range	0.6 MHz to 14 MHz
T_{ADC} range	$1.667\ \mu\text{s}$ to $0.0714\ \mu\text{s}$
Sources (<code>CKMODE</code> in <code>ADC_CFGR2</code>)	dedicated 14 MHz clock, bus clock $\div 2$, bus clock $\div 4$

Note there is a **minimum** as well as a maximum. Run the ADC too slowly and the hold capacitor droops during the conversion, so the later bits of the binary search compare against a decayed voltage. Analog circuits have lower speed limits in a way digital ones do not.

7.3 Channels

The book's part offers 19 channels through the mux, including three special internal ones:

- **Channel 16:** on-chip temperature sensor.
- **Channel 17:** internal reference V_{REFINT} , nominally 1.23 V.
- **Channel 18:** half the voltage on the external V_{BAT} pin, via a divider.

These need enabling in the ADC Common Configuration Register, `ADC_CCR`.

Each channel x has a `CHSELx` bit in `ADC_CHSELR`; set it to include that channel. Set several and the ADC scans the list, converting each in turn.

V_{REFINT} is more useful than it looks

An internal reference of known voltage lets you *measure your own* V_{DDA} . Convert V_{REFINT} , and since you know it is really 1.23 V, whatever code comes back tells you what your reference voltage actually is:

$$V_{DDA} = 1.23 \times \frac{2^B - 1}{\text{code}}$$

That matters because every other reading you take is scaled by V_{DDA} , and on a USB-powered board V_{DDA} is not exactly 3.3 V. Recall the resolution-versus-accuracy trap from Section 1: this is how you claw back some of that accuracy.

And the step that catches everyone: to be used as an ADC input, the GPIO must be in **analog mode**, meaning `MODER` set to 11. Recall from Week 03 that this is the fourth setting of the first pin multiplexer, and recall from Week 07 that analog mode *disables the pin's input buffer*, which is why an analog pin cannot also generate an EXTI interrupt.

7.4 Conversion time

$$T_{conv} = T_{SMPL} + T_{SAR}$$

The SMP field sets sampling time in ADC clock cycles, from 1.5 cycles (SMP=000) to 239.5 cycles (SMP=111). At $f_{ADC} = 14$ MHz that is $0.107 \mu s$ to $17.107 \mu s$.

The RES field in ADC_CFGR1 sets resolution, and **as Section 4 predicted, lower resolution means fewer SAR cycles and a shorter T_{SAR} .**

So the sample rate you can achieve is a three-way negotiation: resolution against speed, and sample time against source impedance. Worth working an example. At $f_{ADC} = 14$ MHz, 12-bit resolution, and SMP = 1.5 cycles, T_{conv} is roughly $1.5 + 12.5 = 14$ cycles = $1 \mu s$, giving about 1 Msps. Raise SMP to 239.5 cycles for a high-impedance source and you get roughly 55 ksps from the same hardware. **A factor of eighteen, decided by one register field.**

Book board vs your board

Your ADC is a substantially better instrument than the book's, and this is where it matters most.

	Book: STM32F091RC	Yours: STM32F303VCT6
ADC units	1	4 (ADC1/2 pair, ADC3/4 pair)
Max rate	~1 Msps	up to 5 Msps
Resolution	12-bit	12-bit
Differential input	No	Yes
Comparators	2	7
Op-amps	—	4
DAC channels	1	2
FPU / DSP	No	Yes

Three consequences worth drawing out.

Four ADCs means genuinely simultaneous sampling. Recall Section 5.1: one ADC scanning channels samples them at different instants. With ADC1 and ADC2 as a pair you can sample two signals at the same moment, which matters for anything computing a relationship between two quantities.

Differential input in hardware means noise rejection without the software subtraction and without the timing skew.

The FPU and DSP instructions are the reason Lab 07 can ask you to run `arm_mean_f32` on `float32_t` at all. On the book's Cortex-M0 every floating-point operation is a software library call. Your Cortex-M4F does it in hardware, which is what makes real-time filtering practical.

In the lab

Mapping this onto what you actually built in Lab 07.

Task 1: potentiometer wired as a three-terminal variable divider, one end to 3.3 V, the other to ground, wiper to the ADC pin. That is Section 1 made physical: rotating the knob sweeps V_{in} across the full reference range, and you confirmed the full 0–4095 span, which is exactly 2^{12} codes.

Task 2: function generator into **PA1**, which is **ADC1_IN2** on the F303, with a 1.65 V DC offset and 1.5 V peak-to-peak. Note why that offset was necessary and not arbitrary: the

ADC range is 0 to 3.3 V, so a sine centred on 0 V would have its entire negative half clipped to code 0. Centring on 1.65 V, exactly half the reference, puts the whole waveform inside the range. That is a real design step, and **"why the 1.65 V offset" is a very plausible viva question.**

Your polling version spins on the completion flag; your interrupt version does everything, ADC read, filter, and UART transmission, inside `HAL_ADC_ConvCpltCallback`. Recall the Week 08 partitioning discussion: doing the UART transmission inside the callback is a **long ISR**. It works here because nothing else is competing, and it is the pattern to reconsider once your project has a control loop with a deadline.

And that is a wrap on the analog world. Timers next, which is where the hardware trigger from Section 5.4 comes from.

8 Tying it back

We asked whether an MCU can distinguish 10.000 °C from 10.001 °C, and the answer turned out to require the whole chapter.

Converting analog to digital costs you information on two independent axes. On the value axis, **quantization** chops the range into 2^B levels, so each code represents a band of voltages that are thereafter indistinguishable, with a step of $V_{ref}/2^B$ and a worst-case error of half that. At 12 bits and 3.3 V that is 0.806 mV and ± 0.403 mV, which is sixteen times too coarse for our temperature question, so the answer is no and the fix is more gain rather than more bits.

On the time axis, **sampling** discards everything between samples, and it fails in a categorically worse way: below $f_s > 2f_h$ you get **aliasing**, where high frequencies impersonate low ones convincingly and irreversibly. So you oversample and you fit an *analog* anti-aliasing filter before the converter, and you accept that a digital filter afterwards, like your Lab 07 moving average, smooths noise but cannot undo an alias.

The converter itself is almost always **successive approximation** rather than flash, because one comparator taking B steps beats 2^B comparators taking one, which is also why lower resolution converts faster. Around it sit a **sample and hold** whose capacitor needs a minimum charging time that grows with source impedance, an **input multiplexer** that makes multi-channel sampling sequential rather than simultaneous, and a **trigger** that should come from a timer rather than from software if you want a sample rate you can trust.

Which is the perfect handoff. Everything above assumed a regular clock and a hardware trigger, and neither has been explained. Next week: SysTick, the general-purpose timers, and how to generate a periodic interrupt whose period is exact.

Cheat sheet: Week 11

Quantization, the value axis

2^B levels for B bits. Two references V_{ref+} , V_{ref-} bound the range.

Each code represents a **range of voltages, which are therefore indistinguishable**.

$$V_{n_lower} = \frac{n}{2^B}(V_{ref+} - V_{ref-}) \quad V_{n_upper} = \frac{n+1}{2^B}(V_{ref+} - V_{ref-})$$

Transfer function: $n = \text{round}\left(2^B \frac{V_{in} - V_{ref-}}{V_{ref+} - V_{ref-}}\right)$ Code to voltage: $D = \frac{V_{in}}{V_{ref}}(2^B - 1)$

Step: $\Delta V = \frac{V_{ref}}{2^B}$ **Max error:** $\pm \frac{\Delta V}{2} = \pm \frac{V_{ref}}{2^{B+1}}$ (half a step, worst case is a boundary)

At $V_{ref} = 3.3 \text{ V}$: **8-bit** 12.89 mV / $\pm 6.45 \text{ mV}$ · **10-bit** 3.22 mV / $\pm 1.61 \text{ mV}$ · **12-bit** 0.806 mV / $\pm 0.403 \text{ mV}$

Each extra bit halves the step, so $\Delta V \propto 2^{-B}$.

Resolution $\neq$ **accuracy**. Accuracy also involves reference, gain, offset, nonlinearity, noise.

Sampling, the time axis

Everything between samples is lost. **Nyquist:** need $f_s > 2f_h$.

Sampling makes **copies of the spectrum** at multiples of f_s . Too slow $\Rightarrow$ they overlap $\Rightarrow$ **aliasing**.

The key asymmetry: low resolution gives a *slightly wrong* answer bounded by half an LSB. Low sample rate gives a *confidently wrong* answer with no warning, and it is **irreversible** — no later filtering can undo it.

Two defences, use both: **(1)** f_s well above $2f_h$, **(2)** an **analog** low-pass anti-aliasing filter **before** the ADC. Extra headroom buys a gentler, cheaper filter.

Architectures

Flash: 2^B comparators in parallel, **one** time step. Each extra bit **doubles** the comparators. Very fast, very expensive. Not in MCUs.

SAR: one comparator + DAC + SAR + control logic, doing a **binary search** in B steps. Cheap, and **fewer bits = fewer steps = faster**. This is what your MCU has.

A **comparator** is a 1-bit ADC and can interrupt on a threshold without any polling.

Around the converter

Input mux: one ADC serves many channels, but **sequentially**, not simultaneously.

Sample and hold: switch + capacitor. Holds the input steady so the SAR search compares against one value. **Minimum sample time**, and it grows with *source impedance* — a high-impedance sensor with a short SMP reads systematically low.

Single-ended = signal vs ground. **Differential** = signal vs signal, rejects common-mode noise.

Triggers: software (write ADSTART) or hardware (a timer). **Use a timer for uniform f_s** ; software triggering jitters with every branch and interrupt.

On completion: status flag set, optional interrupt, result in the data register.

The peripheral

ADC_CR: ADEN enable, ADDIS disable, ADSTART allow conversions, ADSTP stop, ADCAL calibrate.

After ADEN, **poll** ADRDY in ADC_ISR. Analog peripherals need warm-up; skipping this makes the first readings garbage.

References: V_{DDA} (3.3 V nominal) and V_{SSA} .

f_{ADC} has a **minimum as well as a maximum** (0.6–14 MHz on the F091) because the hold capacitor droops if conversion is too slow.

GPIO must be in analog mode, $MODER = 11$ — which also disables its input buffer, so no EXTI.

Special channels: **16** temperature sensor, **17** V_{REFINT} (≈ 1.23 V), **18** $V_{BAT}/2$. Enable via ADC_CCR .

V_{REFINT} lets you measure your own V_{DDA} : $V_{DDA} = 1.23 \times \frac{2^B - 1}{\text{code}}$.

$T_{conv} = T_{SMPL} + T_{SAR}$. SMP sets 1.5 to 239.5 ADC cycles; RES sets resolution and thus T_{SAR} .

Your board vs the book's

4 ADCs (vs 1) — ADC1/2 and ADC3/4 pairs give **genuinely simultaneous** sampling.

Up to 5 Msps (vs ~ 1). **Differential input in hardware**. **7 comparators**, **4 op-amps**, **2 DACs**.

FPU and DSP instructions — the reason `arm_mean_f32` on `float32_t` is practical at all. On the M0, every float operation is a library call.

Op-amps can amplify a weak sensor *before* the ADC, which is the real fix for insufficient resolution.

Likely exam prompts from this chapter

- Given V_{ref} and B , find the step size and maximum quantization error. *Near certain.*
- Explain the relationship between resolution and quantization error, and why more bits reduce it. *This is your lab's own evaluation question.*
- State the Nyquist criterion and explain aliasing. Why can a digital filter not fix it?
- Compare flash and SAR architectures, with the cost and time scaling.
- Why does an ADC need a sample-and-hold circuit, and what sets the minimum sample time?
- Why prefer a hardware timer trigger over a software trigger?
- Why must an ADC input pin be in analog mode, and what does that cost you?